	SKT SOFTWARE SOLUTION	CÓDIGO: MI-GAM-PRO-001
SKT SOFTWARE SOLUTION	PROCESO: DESARROLLO DE SOFTWARE	VERSIÓN: 1.0
	Arquitectura de Base de Datos	ELABORACIÓN: 24/03/2026
	Modelo Relacional (DER) y Diccionario de Datos	ÚLTIMA REVISIÓN: 24/03/2026

1. Título del Documento

Especificación Técnica del Modelo Relacional de Base de Datos (DER), Diccionario de Datos y Políticas de Seguridad Avanzada (RLS/RBAC) para el Sistema LUPSI

2. Resumen

El presente documento constituye el entregable técnico formal que define la arquitectura de datos relacional para el proyecto LUPSI. El modelado, implementado sobre PostgreSQL (Supabase), trasciende la mera definición de tablas; establece un framework de integridad referencial, control de concurrencia y seguridad de nivel militar (Row Level Security).

Este diseño garantiza de forma nativa la solución a los dolores de negocio identificados: elimina el *double-booking* mediante bloqueos transaccionales, asegura la veracidad de los pacientes mediante restricciones algorítmicas y blinda la privacidad de las historias clínicas aislando los datos sensibles del personal administrativo, cumpliendo estrictamente con el presupuesto de infraestructura de $150 USD.

3. Descripción General del Diagrama Entidad-Relación (DER)

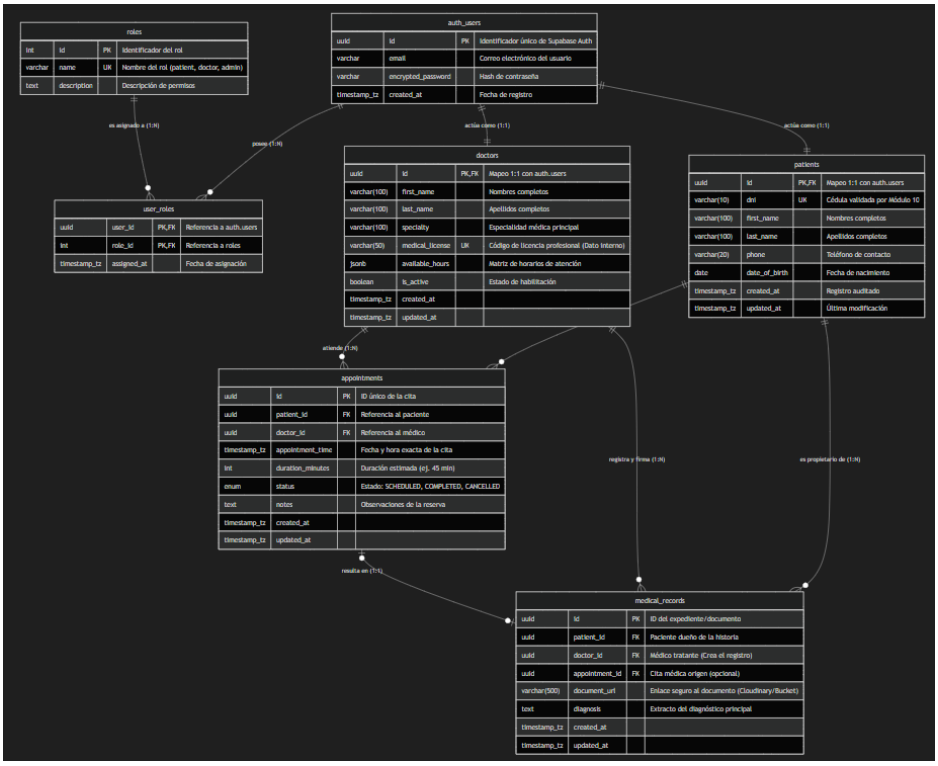


Figura 1: Diagrama Entidad-Relación (DER) del Sistema LUPSI. Se observa la normalización en 3NF y la separación nítida entre esquemas de autenticación y negocio.

El modelo sigue una arquitectura híbrida BaaS/IaaS. Se apoya en el esquema auth nativo de Supabase...`auth` nativo de Supabase para la gestión de identidad segura y define un esquema público personalizado para la lógica de negocio. El diseño prioriza la integridad referencial fuerte y el uso de tipos de datos avanzados de PostgreSQL (UUID, JSONB, TIMESTAMPTZ) para maximizar el rendimiento y la flexibilidad.

4. Diccionario de Datos Detallado y Especificaciones Técnicas

A continuación, se describen las entidades críticas, desglosando atributos, tipos de datos, restricciones y la justificación técnica de su diseño.

4.1. 1. Esquema de Identidad y Autorización (RBAC)

La seguridad se gestiona mediante un modelo de Control de Acceso Basado en Roles (RBAC), definido en tablas transaccionales que alimentan los *Guards* de NestJS.

Tabla	Atributo / Tipo	Restricció	Descripción Técnica y Justificación
`roles`	`id` (PK) / UUID `name` / VARCHAR	NOT NULL UNIQUE	Define los roles estáticos del sistema (*patient, doctor, receptionist*). Se usa VARCHAR con UNIQUE para garantizar la integridad semántica de los roles.
`user_roles`	`user_id` (FK) / UUID `role_id` (FK) / UUID	PK(composi NOT NULL	Tabla puente para relación N:N. Asocia usuarios de Supabase con roles. PK compuesta para evitar duplicidad de roles por usuario. Crucial para la auditoría de accesos.

4.2. 2. Esquema de Negocio: Perfiles y Operaciones Core

Tabla	Atributo / Tipo	Restricción	Descripción Técnica y Justificación
`patients`	`id` (PK/FK) / UUID `dni` / VARCHAR(10) `birth_date` / DATE	UNIQUE (dni) CHECK (age)	Extiende el perfil de `auth.users`. `dni` UNIQUE blinda contra registros duplicados. El backend NestJS validará el Módulo 10 antes de permitir el `INSERT`. El campo DATE permite calcular la edad dinámicamente sin almacenarla, respetando la 3NF.
`doctors`	`id` (PK/FK) / UUID `available_hours` / JSONB	NOT NULL (JSONB)	Almacena la matriz de disponibilidad semanal. Se usa JSONB para permitir estructuras de horarios complejas y variables (ej. turnos partidos) sin crear tablas adicionales, optimizando el rendimiento de lectura de la PWA.
`appointments`	`id` (PK) / UUID `time` / TIMESTAMPTZ `status` / ENUM	NOT NULL DEFAULT ('pending')	Módulo crítico. `TIMESTAMPTZ` garantiza precisión ante cambios de zona horaria. El backend implementará bloqueos transaccionales sobre esta tabla para gestionar la concurrencia y eliminar el *double-booking*.

4.3. 3. Esquema de Datos Sensibles: Historias Clínicas

Tabla	Atributo / Tipo	Restricció	Descripción Técnica y Justificación
`medical_records`	`id` (PK) / UUID `diagnosis` / TEXT `document_url` / TEXT	NOT NULL DEFAULT (”)	Esta tabla concentra el riesgo de privacidad. `document_url` almacena únicamente la referencia de texto hacia Cloudinary (CDN externo). No se almacenan binarios (BLOB/BYTEA) en PostgreSQL para proteger la performance de la base de datos y cumplir el presupuesto.

5. Restricciones de Integridad y Lógica de Negocio Avanzada

Para garantizar que la base de datos sea un "origen único de la verdad" confiable, se han implementado restricciones que trascienden las Claves Primarias y Foráneas:

- **Validación Algorítmica de Identidad (Módulo 10):** Aunque la lógica reside en el backend NestJS (CoreModule), la base de datos soporta esta regla mediante una restricción `UNIQUE` en el campo `patients.dni`. Esto garantiza que un DNI validado matemáticamente no pueda ser reutilizado, eliminando las "citas fantasma" por suplantación.
- **Motor de Concurrencia y Double-Booking:** La tabla `appointments` es el sujeto de bloqueos transaccionales por parte de NestJS. Al intentar agendar, NestJS abrirá una transacción PostgreSQL y bloqueará la fila del médico y el slot de tiempo. Si la transacción falla por concurrencia, el sistema rechazará la escritura, garantizando que el médico nunca tenga dos pacientes en el mismo milisegundo.
- **Gestión de Almacenamiento Híbrido:** Se ha establecido una política estricta de "cero binarios en base de datos". La tabla `medical_records` solo

acepta URLs validadas apuntando a Cloudinary, manteniendo la base de datos ligera, rápida y dentro de los límites de costo operativo.

6. Modelo de Seguridad Advanced: Row Level Security (RLS)

Para cumplir con el requerimiento del médico de aislar los antecedentes patológicos del personal de recepción, se habilitará la Seguridad a Nivel de Fila (RLS) directamente en PostgreSQL. Esto blinda los datos incluso si un atacante logra acceso directo a la API.

1. **Tabla `appointments`:** Se definirán políticas RLS que permitan `SELECT` e `UPDATE` al rol *receptionist* para la gestión operativa.

2. **Tabla `medical_records`: El acceso está denegado globalmente** para el rol *receptionist*. Las políticas RLS (`SELECT, INSERT`) solo se activarán si el UUID del token autenticado coincide con el `doctor_id` asignado a la cita o el `patient_id` propietario del diagnóstico, garantizando el aislamiento absoluto de los datos clínicos.

	SKT SOFTWARE SOLUTION	CÓDIGO: MI-GAM-PRO-001
	PROCESO: DESARROLLO DE SOFTWARE	VERSIÓN: 1.0
	Arquitectura de Base de Datos	ELABORACIÓN: 24/03/2026
	Modelo Relacional (DER) y Diccionario de Datos	ÚLTIMA REVISIÓN: 24/03/2026

7. Firmas de Responsabilidad

Rol	Nombre / Representante	Firma (QR)	Fecha
Gestor del Proyecto	Angel Ayuquina		24/03/2026
Arquitectura y BD	Sebastián Ortiz		24/03/2026
Desarrollador Backend	Daniel Luisa		24/03/2026
Desarrollador Frontend	Alex Guachi		24/03/2026

8. Control de Cambios

Versión	Fecha	Descripción del Cambio	Responsable
1.0	24/03/2026	Especificación técnica del modelado relacional, diccionario de datos avanzado y diseño de políticas RLS para Supabase.	Sebastián Ortiz